课程笔记

CS336 第 6 讲：Kernel、Triton 与 “让 GPU 真正跑快”

Codex 基于 Stanford CS336 2025 第 6 讲视频、字幕与官方课程材料重写

2026-04-07

视频作者/频道：Stanford Online
发布日期：2025 年 05 月 08 日
视频时长：1:20:22
视频链接：`https://www.youtube.com/watch?v=E8Mju53VB00`

目录

1 这讲在系统主线中的位置 **2**

1.1 从 A100 / H100 规格到真实 workload 2

1.2 本章小结 2

2 先别优化，先 benchmark / profile **2**

2.1 为什么 benchmarking 比“看 spec sheet”更重要 2

2.2 Benchmark 解决的是“有多快”，profiling 解决的是“快慢来自哪” 2

2.3 为什么矩阵乘法看起来简单，性能却并不总是可预测 3

2.4 本章小结 3

3 Arithmetic Intensity：把所有优化语言统一起来 **3**

3.1 为什么课程要把“算术强度”放在整讲中央 3

3.2 为什么矩阵乘法通常是 compute-bound，而很多别的操作不是 4

3.3 本章小结 4

4 六种常见提速手段：本质都在和 memory 搏斗 **4**

4.1 低精度：先从少搬一点数据开始 4

4.2 operator fusion：不要把仓库物流当成默认代价 4

4.3 recomputation：有时重算比存下再读更划算 5

4.4 coalescing 与 tiling：真正的大头 6

4.5 本章小结 6

5 CUDA 与 Triton：为什么课程要你往下看到 kernel 代码 **6**

5.1 为什么手写一个简单 CUDA GeLU 仍然很有价值 6

5.2 Triton 的真正价值：把大量底层机械活自动化，但不抹掉硬件思维 6

5.3 本章小结 6

6 总结与延伸 **7**

1 这讲在系统主线中的位置

第 5 讲已经解释了 GPU 不是魔法，而是一种受 memory hierarchy 强烈约束的吞吐机器。第 6 讲继续往下一层走：既然我们已经知道 GPU 为什么会快、为什么会慢，那么**怎样真正写出或理解一个高性能 kernel？**这就是这讲的中心。

课程把任务拆成两半：一半是 benchmarking / profiling，另一半是 CUDA / Triton 视角下的 kernel 编写。这个拆法很合理，因为你不先知道“慢在哪里”，就没有资格讨论“该怎么改快”。

第 6 讲的中心命题

高性能 kernel 不是“把一段数学公式翻译成 GPU 代码”那么简单，而是要把计算组织成一种符合 GPU execution model 与 memory hierarchy 的形式。

1.1 从 A100 / H100 规格到真实 workload

课程在最开始先回顾硬件：SM 数量、DRAM、L2、L1 / shared memory 等。这里的重点并不是让学生背规格，而是建立一种“现实硬件是有限资源池”的直觉。即使你有 H100，shared memory 仍然很小；即使 matmul 非常快，global memory 仍然是瓶颈；即使你能开很多线程，thread blocks 仍然要按 SM 分波次调度。

1.2 本章小结

这一讲不是在重复 GPU 课，而是在为 kernel 设计建立实践入口：你不只是需要知道 GPU 有什么组件，还要知道这些组件会怎样真实限制你的程序。

2 先别优化，先 benchmark / profile

2.1 为什么 benchmarking 比“看 spec sheet”更重要

课程对 benchmarking 的态度很明确：你当然可以看论文、看 spec、看厂商材料，但对自己手头的库版本、硬件和 workload 来说，**没有任何东西能替代直接 benchmark / profile**。这是因为 kernel 性能高度依赖具体实现、输入形状、数据类型和运行环境。

这一点和第 2 讲的 resource accounting 一脉相承。第 2 讲给你的是一阶近似和数量级判断；第 6 讲要你开始接受另一个现实：很多运行时现象并不是纸面 FLOPs 直接决定的，而必须靠实测来验证。

2.2 Benchmark 解决的是“有多快”，profiling 解决的是“快慢来自哪”

课程区分得很清楚：

- **benchmarking** 给的是 end-to-end wall-clock time。
- **profiling** 则帮你看清具体哪些 op、哪些 kernel、哪些同步点在吃时间。

这个区分非常重要。只 benchmark 不 profile，你只能知道 A 比 B 快，却不知道原因；只 profile 不 benchmark，你可能知道热点在哪，却不知道这些热点对整体吞吐的实际贡献有多大。

2.3 为什么矩阵乘法看起来简单，性能却并不总是可预测

课程专门用 square matmul 和一个小 MLP 做 benchmark，随后演示不同维度、层数、batch size 下时间变化并不总是线性可预测。这再次说明：真实 CUDA kernel 会受到 tile 形状、调度、对齐、occupancy 和库内实现细节的影响。

关于 kernel 性能的一个常见误区

“数学上复杂度一样，所以速度应该差不多”在 GPU 上经常不成立。相同数量级的操作，如果访存模式、tile 对齐或 kernel 调度方式不同，最终时间可能差很多。

2.4 本章小结

优化之前先 benchmark / profile，并不是保守做法，而是唯一靠谱的起点。因为你要面对的不是抽象程序，而是某个具体环境里的某个具体 kernel。

3 Arithmetic Intensity：把所有优化语言统一起来

3.1 为什么课程要把“算术强度”放在整讲中央

第 6 讲最重要的统一指标就是 arithmetic intensity：

$$I = \frac{\#\text{FLOPs}}{\#\text{Bytes}}$$

课程给出的解释非常直接：如果这个比值高，操作更可能是 compute-bound；如果这个比值低，操作更可能是 memory-bound。它之所以重要，是因为几乎所有 kernel 优化技巧，最终都可以翻译成“怎样提高或保护算术强度”。

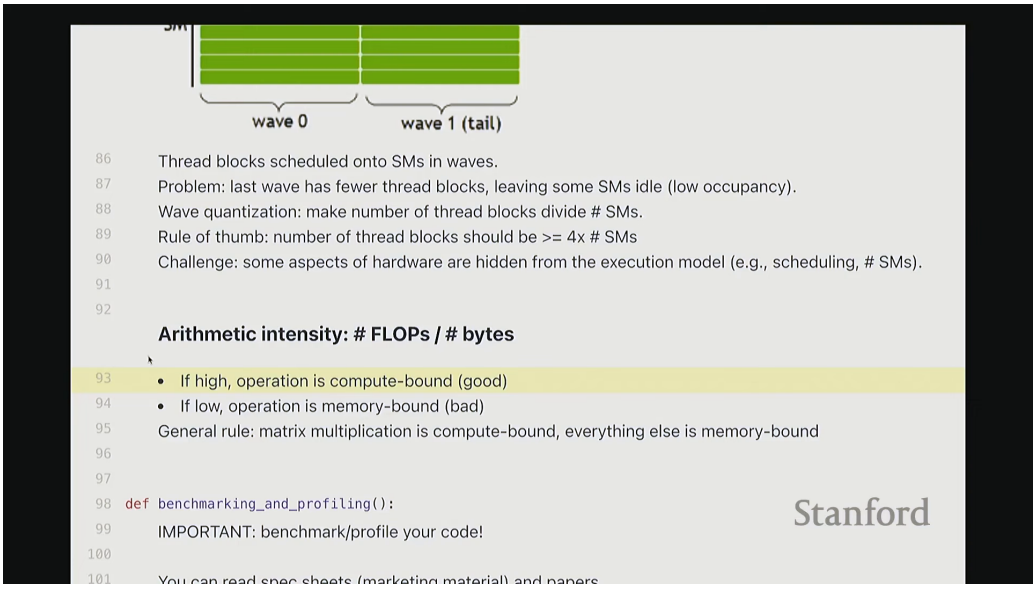

图 1: 课程直接把 arithmetic intensity 放到中间位置：高表示更接近 compute-bound，低表示更容易被 memory-bound 卡死。[1]

[1] 视频画面时间区间：约 00:07:40–00:08:00。该图来自课程视频中的屏幕画面。

3.2 为什么矩阵乘法通常是 compute-bound，而很多别的操作不是

矩阵乘法之所以特殊，是因为一次搬进来的数据会被大量重复用于乘加，因此更容易把计算做“厚”；而 elementwise、简单激活、归一化之类的操作，往往每搬一次数据只做很少计算，因此更容易变成 memory-bound。

这就是后面 operator fusion、tiling、shared memory 这些技巧会围着 matmul 以外的操作打转的原因：它们要么试图把本来低强度的操作变得更厚一点，要么至少别让这些操作把整体 pipeline 拖得太惨。

第 6 讲最值得背下来的判断

矩阵乘法通常是 compute-bound，很多其余深度学习常见 op 更容易 memory-bound。理解这条经验，会让后面很多 kernel 设计取舍立刻变得顺理成章。

3.3 本章小结

arithmetic intensity 在这讲里起到的是“元语言”作用。它让你不用把每个技巧都孤立记忆，而是统一用“更多计算 / 更少搬运”的尺度去理解它们。

4 六种常见提速手段：本质都在和 memory 搏斗

4.1 低精度：先从少搬一点数据开始

课程再次强调低精度的重要性，但这次重心不是训练稳定性，而是 kernel 性能。位宽下降意味着一次内存传输带来的元素数增加，从而直接改善很多操作的算术强度。尤其在 tensor cores 已经高度优化低精度矩阵乘法的情况下，这种收益非常直接。

4.2 operator fusion：不要把仓库物流当成默认代价

课程用了一个非常好的类比：memory 像仓库，compute 像工厂。如果每做一个小操作都把中间结果运回仓库，再读出来做下一步，那整体性能很可能被物流拖死。fusion 的意义就是把本来会反复读写 global memory 的连续操作合进一个 kernel。

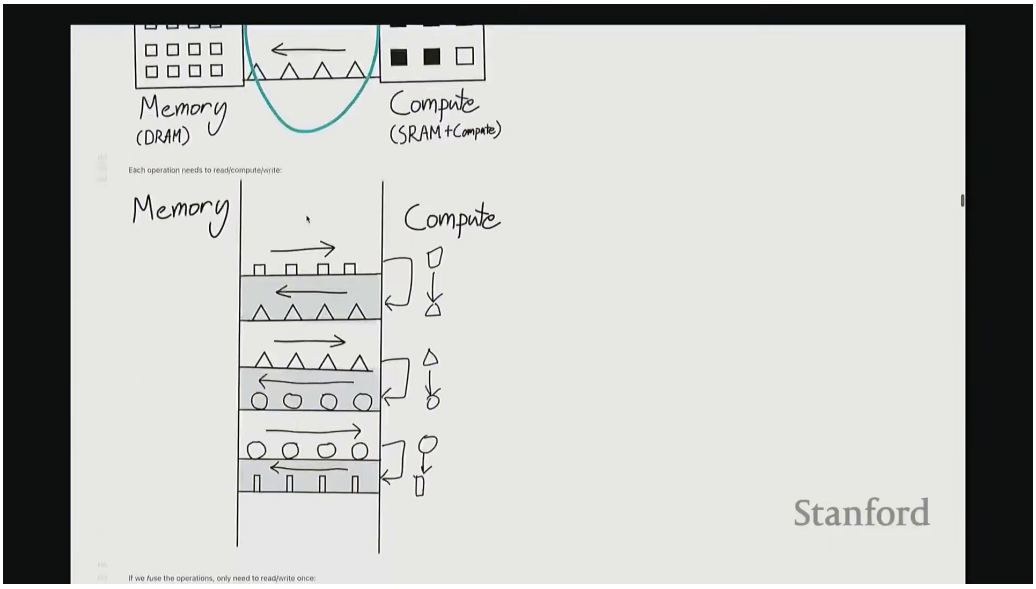

图 2: memory 是仓库，compute 是工厂。这个类比几乎可以概括第 6 讲对 kernel 优化的全部直觉。

图 3: operator fusion 的收益来自减少中间结果反复写回 / 读出 global memory，而不是来自“数学更优雅”。[2]

课程用 GeLU 举例非常典型。手写版本会拆成多次 elementwise op，每一步都可能触发独立 kernel；PyTorch 的 fused 版本则只需要更少 kernel 调用。这种差异在 GPU 上会非常大，因为你节省的是 launch 开销和 memory round-trip。

4.3 recomputation：有时重算比存下再读更划算

第 6 讲延续了第 5 讲对 activation checkpointing 的直觉，但放得更通用：有时候保留所有中间结果会导致巨大的 memory 读写开销，倒不如在需要的时候局部重算。只要重算的 FLOPs 比搬运的代价更便宜，这笔账就可能是合算的。

[2] 视频画面时间区间：约 00:42:00–00:42:20。该图来自课程视频中的屏幕画面。

4.4 coalescing 与 tiling：真正的大头

coalesced memory access 和 tiling 可以看作这讲最“硬核”的两部分。前者要求同一 warp 的线程尽量整齐地访问连续内存，从而充分利用 burst mode；后者则通过把矩阵切成 tile 并搬进 shared memory，减少对 global memory 的重复读取。

tiling 为什么重要？因为在非 tiled 矩阵乘法里，很多数据会被反复从远处读进来；一旦 tiled，每个 tile 可以在更近的 shared memory 中被重复利用，从而把 global memory 访问次数压低一个数量级。

为什么 tiling 是“大招”

tiling 不是一个孤立 trick，而是把“尽量在快存储里重复利用数据”这个总原则写成程序结构的方式。很多高性能 matmul、本地 attention 和 fused kernel，本质上都靠它在撑。

4.5 本章小结

这部分最关键的 takeaway 是：低精度、fusion、recomputation、coalescing、tiling 等技巧并不分裂，它们都在围绕同一个目标工作：让计算更厚，让搬运更少，让数据尽量待在更近的存储层里。

5 CUDA 与 Triton：为什么课程要你往下看到 kernel 代码

5.1 为什么手写一个简单 CUDA GeLU 仍然很有价值

课程先让学生看一个简单的 CUDA GeLU 实现，这件事的意义不在于“未来你必须全部手写 CUDA”，而在于把抽象层扒开，让你看到 kernel 最底层到底在做什么：线程索引怎样决定处理哪个元素、grid / block 怎样映射、边界怎样处理、输出张量怎样写回。

这一步很重要，因为只有理解过这层结构，你才会真正知道 PyTorch fused op 或 Triton kernel 到底替你自动完成了哪些事情。

5.2 Triton 的真正价值：把大量底层机械活自动化，但不抹掉硬件思维

课程对 Triton 的定位非常准确。它不是让你“忘掉硬件”，而是让你以更高层的方式表达 kernel，同时把 memory coalescing、shared memory 管理、部分调度细节交给编译器。也就是说，**Triton 降低了进入 kernel 世界的门槛，但并没有取消对 hardware-aware 思维的要求**。

为什么 Triton 特别适合 CS336

这门课想让学生真正理解 kernel，但又不想让所有人都先被 CUDA 语法细节淹没。Triton 恰好处在一个好位置：你仍然在思考 block、tile、访存与布局，但可以用更高层的写法落地。

5.3 本章小结

第 6 讲最后从 benchmarking/profiling 一路走到 CUDA 和 Triton，其实是在完成一个非常完整的闭环：先测哪里慢，再理解慢的根源，最后把这种理解变成更好的 kernel 设计。

6 总结与延伸

第 6 讲最重要的结论可以压缩成四点。

第一，**benchmark / profile 是 kernel 优化的起点，而不是收尾**。没有实测，你只是猜。

第二，**arithmetic intensity 是统一理解 kernel 优化的核心尺度**。高强度意味着更接近 compute-bound，低强度则意味着你更容易被 memory 拖死。

第三，**GPU 提速技巧的共同本质，是减少或重组数据搬运**。fusion、recomputation、coalescing、tiling 全都属于这个框架。

第四，**Triton 的价值不在于隐藏硬件，而在于让你能用更合理的抽象直接表达 hardware-aware kernel**。

我对第 6 讲的压缩提醒

如果你还把 kernel 优化理解成“把公式翻译成更底层代码”，那你会一直抓不到重点。真正的重点是：你有没有把数据组织成 GPU 愿意高效处理的形状。

这讲之后，自然要进入并行训练与更复杂的系统层，因为单个 GPU 上的 kernel 理解，最终会扩展成多卡、多机上的通信与分片问题。也就是说，第 6 讲是从“硬件直觉”走向“分布式系统直觉”的最后一站。